

University of Glasgow

LAB 3

Real-Time Computing Systems & Architecture

Real Time Operating Systems (RTOS)

Name: Zhanyu Zhang
UoG ID: 2959456
UESTC ID: 2023300901013

Abstract

This laboratory implements task scheduling, inter-task communication, and stack allocation using CMSIS-RTOS2. The first application assigns a higher priority to a button-controlled brake indicator than to its UART notification. The second application uses two tasks and three queues to process eleven-character commands, control an LED, count toggles, and report the maximum of 150 random integers. Hardware tests confirmed the button response, command echo, and queue updates. Mean LED toggle intervals were 1006.0, 502.9, and 251.4 ms for commands of 1000, 500, and 250 ms, respectively.

1 Introduction

An RTOS schedules concurrent tasks according to their priorities and ready states. Thread flags communicate events, while message queues transfer values between tasks. Each task also requires a stack sized for its local data and function calls. These mechanisms are examined through a brake indication task and a UART-controlled LED task. The RTOS tick is 1 ms, and TIM4 provides a separate HAL time base.

2 Task 1: Brake Indication and UART Notification

Pressing B1 generates a falling-edge interrupt. The callback applies a 30 ms debounce interval and sets the brake task's thread flag. The brake task switches LD2 on, then signals the message task to transmit `Car is slowing down`. It checks for button release every 10 ms and switches LD2 off when B1 is released.

Table 1: Task 1 priorities and stack allocation.

Task	CMSIS priority	Stack (bytes)	Function
Brake	High	512	Apply the brake indicator
Message	Normal	768	Transmit the notification

Brake actuation has higher priority because it is the immediate response to the slowdown signal. The LED is set before the UART task is notified. Once the brake task blocks during its release check, the message task can transmit. Both tasks wait on flags between events.

The hardware test showed LD2 illuminated while B1 was held and extinguished after release. Figure 1 shows the UART messages received during repeated presses.

```
Car is slowing down
Car is slowing down
Car is slowing down
Car is slowing down
Car is slowing down
Car is slowing down
Car is slowing down
Car is slowing down
```

Figure 1: UART notifications generated by B1 presses.

2.1 Implementation

Listing 1 contains the two tasks, their creation, and the external interrupt callback. Thread flags connect the interrupt, brake action, and notification.

Listing 1: Task 1: priority-based brake indication and messaging.

```c
#include "lab3.h"

#if LAB3_TASK == 1

#define BUTTON_EVENT 1U
#define MESSAGE_EVENT 1U

static osThreadId_t brake_thread;
static osThreadId_t message_thread;

static void BrakeTask(void *argument)
{
  (void)argument;

  for (;;) {
    osThreadFlagsWait(BUTTON_EVENT, osFlagsWaitAny, osWaitForever);

    /* Apply the brake before waking the lower-priority UART task. */
    HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, GPIO_PIN_SET);
    osThreadFlagsSet(message_thread, MESSAGE_EVENT);

    /* Keep the brake applied while the button is held. */
    while (HAL_GPIO_ReadPin(B1_GPIO_Port, B1_Pin) == GPIO_PIN_RESET) {
      osDelay(10U);
    }
    HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, GPIO_PIN_RESET);
  }
}

static void MessageTask(void *argument)
{
  (void)argument;

  for (;;) {
    osThreadFlagsWait(MESSAGE_EVENT, osFlagsWaitAny, osWaitForever);
    Console_Write("Car is slowing down\r\n");
  }
}

void Task1_Start(void)
{
  const osThreadAttr_t brake_attributes = {
    .name = "Brake", .priority = osPriorityHigh, .stack_size = 512U
  };
  const osThreadAttr_t message_attributes = {
    .name = "Message", .priority = osPriorityNormal, .stack_size = 768U
  };

  message_thread = osThreadNew(MessageTask, NULL, &message_attributes);
  brake_thread = osThreadNew(BrakeTask, NULL, &brake_attributes);
}

void HAL_GPIO_EXTI_Callback(uint16_t pin)
{
  static uint32_t last_press;
  uint32_t now = HAL_GetTick();

  /* A 30 ms debounce gate accepts one falling edge per press. */
  if ((pin == B1_Pin) && (now - last_press >= 30U)) {
    last_press = now;
    osThreadFlagsSet(brake_thread, BUTTON_EVENT);
  }
}

#endif
```

3 Task 2: UART Commands, Queues, and Memory

The UART task receives an eleven-byte frame through `HAL_UART_Receive_IT`. The completion callback sets a thread flag; the task then echoes the frame and processes it. For `-DELAY 1000`, characters 2–6 identify `DELAY`, and characters 8–11 give the decimal interval. The parsed `uint16_t` value is sent to queue 1.

The LED task waits on queue 1 with a timeout equal to the time remaining until the next toggle. A new command updates the interval and generates 150 random integers in a local array. The task scans the array and sends its maximum through queue 3. On each timeout it toggles LD2. Every ten toggles, it increments a cumulative counter and sends that value through queue 2. Queue updates notify the UART task, which prints the received values.

3.1 Queue and Stack Allocation

Table 2: The three application queues, each with capacity for four items.

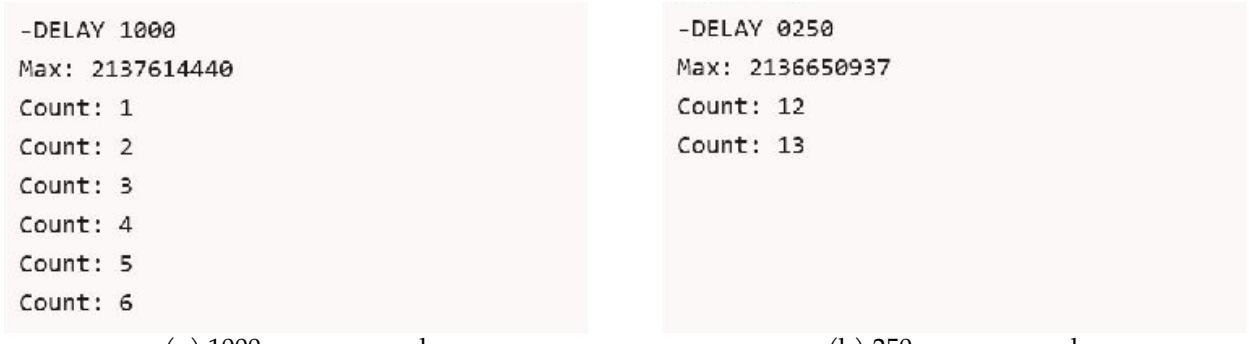
Queue	Item type	Width (bytes)	Payload (bytes)	Direction
1: interval	`uint16_t`	2	8	UART → LED
2: count	`uint32_t`	4	16	LED → UART
3: maximum	`int`	4	16	LED → UART

Four items allow several pending values to be retained while the receiving task runs. Queue 1 uses the required 16-bit interval type. Queue 2 uses a 32-bit cumulative count, and queue 3 matches the array's four-byte integer type. Their combined payload is 40 bytes; FreeRTOS also allocates queue control structures from the 8192-byte heap using `heap_4`.

The local array occupies $150 \times 4 = 600$ bytes. The LED task receives a 1536-byte stack, equivalent to 384 words, for the array, local variables, function calls, and saved context. Its measured minimum unused stack was 192 bytes, corresponding to a maximum observed use of 1344 bytes. The UART task receives a 1024-byte stack and Above Normal priority; the LED task uses Normal priority. This lets completed UART frames and queue results be processed promptly.

3.2 Serial Results

Figure 2 shows commands echoed by the board and the subsequent queue values. Changing the interval produces a new maximum, while the toggle-group counter continues to increase. Leading zeros in `0250` are retained in the echo and converted to the integer 250 for queue 1.



(a) 1000 ms command (b) 250 ms command

Figure 2: Task 2 output: command echo, array maximum, and cumulative count.

3.3 Implementation

The UART task is the sole owner of serial transmission. It rearms reception after consuming the completed frame and drains queues 2 and 3 when notified.

Listing 2: Task 2: command parsing and UART task.

```c
#include "lab3.h"

#include <stdlib.h>
#include <string.h>

#if LAB3_TASK == 2

#define UART_EVENT 1U
#define QUEUE_EVENT 2U
#define FRAME_LENGTH 11U
#define QUEUE_LENGTH 4U

extern UART_HandleTypeDef huart2;
static osThreadId_t uart_thread;
static osThreadId_t led_thread;
static osMessageQueueId_t queue1;
static osMessageQueueId_t queue2;
static osMessageQueueId_t queue3;
static uint8_t frame[FRAME_LENGTH];

/* Read in the debugger after a command to measure unused worker stack. */
volatile uint32_t task2_stack_free;

static void ReceiveCommand(void)
{
  uint16_t interval = 0U;
  uint32_t i;

  /* Echo all eleven received bytes, then end the display line. */
  HAL_UART_Transmit(&huart2, frame, FRAME_LENGTH, 100U);
  Console_Write("\r\n");

  if (memcmp(&frame[1], "DELAY", 5U) != 0) {
    return;
  }
  for (i = 7U; i < FRAME_LENGTH; ++i) {
    if ((frame[i] < '0') || (frame[i] > '9')) {
      return;
    }
    interval = (uint16_t)(interval * 10U + frame[i] - '0');
  }
  if (interval != 0U) {
    osMessageQueuePut(queue1, &interval, 0U, osWaitForever);
  }
}

static void UartTask(void *argument)
{
  uint32_t flags;
  uint32_t count;
  int maximum;
  (void)argument;

  HAL_UART_Receive_IT(&huart2, frame, FRAME_LENGTH);
  for (;;) {
    flags = osThreadFlagsWait(UART_EVENT | QUEUE_EVENT,
                      osFlagsWaitAny, osWaitForever);
    if ((flags & UART_EVENT) != 0U) {
      ReceiveCommand();
      HAL_UART_Receive_IT(&huart2, frame, FRAME_LENGTH);
    }
    while (osMessageQueueGet(queue2, &count, NULL, 0U) == osOK) {
      Console_Number("Count: ", count);
    }
    while (osMessageQueueGet(queue3, &maximum, NULL, 0U) == osOK) {
      Console_Number("Max: ", (uint32_t)maximum);
    }
  }
}
```

The LED task maintains an absolute deadline between toggles. Queue reception can wake it before that deadline to apply a new interval and generate the array. The stack allocation and queue widths are set when the application starts.

Listing 3: Task 2: LED task, queue creation, and receive callback.

```c
static void LedTask(void *argument)
{
  int samples[150];
  int maximum;
  uint16_t interval = 0U;
  uint16_t command;
  uint32_t deadline = 0U;
  uint32_t toggles = 0U;
  uint32_t count = 0U;
  uint32_t timeout;
  int32_t remaining;
  uint32_t i;
  (void)argument;

  srand(HAL_GetTick());
  for (;;) {
    remaining = (int32_t)(deadline - osKernelGetTickCount());
    timeout = interval == 0U ? osWaitForever
            : (remaining > 0 ? (uint32_t)remaining : 0U);

    /* A queue wait wakes either for a new command or the next toggle. */
    if (osMessageQueueGet(queue1, &command, NULL, timeout) == osOK) {
      interval = command;
      for (i = 0U; i < 150U; ++i) {
        samples[i] = rand();
      }
      maximum = samples[0];
      for (i = 1U; i < 150U; ++i) {
        if (samples[i] > maximum) {
          maximum = samples[i];
        }
      }
      osMessageQueuePut(queue3, &maximum, 0U, osWaitForever);
      osThreadFlagsSet(uart_thread, QUEUE_EVENT);
      deadline = osKernelGetTickCount() + interval;
    } else {
      HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);
      deadline += interval;
      if (++toggles == 10U) {
        toggles = 0U;
        ++count;
        osMessageQueuePut(queue2, &count, 0U, osWaitForever);
        osThreadFlagsSet(uart_thread, QUEUE_EVENT);
      }
    }
    task2_stack_free = osThreadGetStackSpace(led_thread);
  }
}

void Task2_Start(void)
{
  const osThreadAttr_t uart_attributes = {
    .name = "UART", .priority = osPriorityAboveNormal, .stack_size = 1024U
  };
  const osThreadAttr_t led_attributes = {
    .name = "LED", .priority = osPriorityNormal, .stack_size = 1536U
  };

  queue1 = osMessageQueueNew(QUEUE_LENGTH, sizeof(uint16_t), NULL);
  queue2 = osMessageQueueNew(QUEUE_LENGTH, sizeof(uint32_t), NULL);
  queue3 = osMessageQueueNew(QUEUE_LENGTH, sizeof(int), NULL);
  uart_thread = osThreadNew(UartTask, NULL, &uart_attributes);
  led_thread = osThreadNew(LedTask, NULL, &led_attributes);
}

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *uart)
{
  if (uart->Instance == USART2) {
    /* The task consumes the completed frame before reception is rearmed. */
    osThreadFlagsSet(uart_thread, UART_EVENT);
  }
}

#endif
```

3.4 Measured LED Timing

PA5 output states were sampled through ST-LINK while the application ran. Twelve successive transitions were recorded for each command, giving eleven inter-toggle intervals. Figure 3 shows the first 3.2 s of each trace, and Table 3 summarizes the intervals.

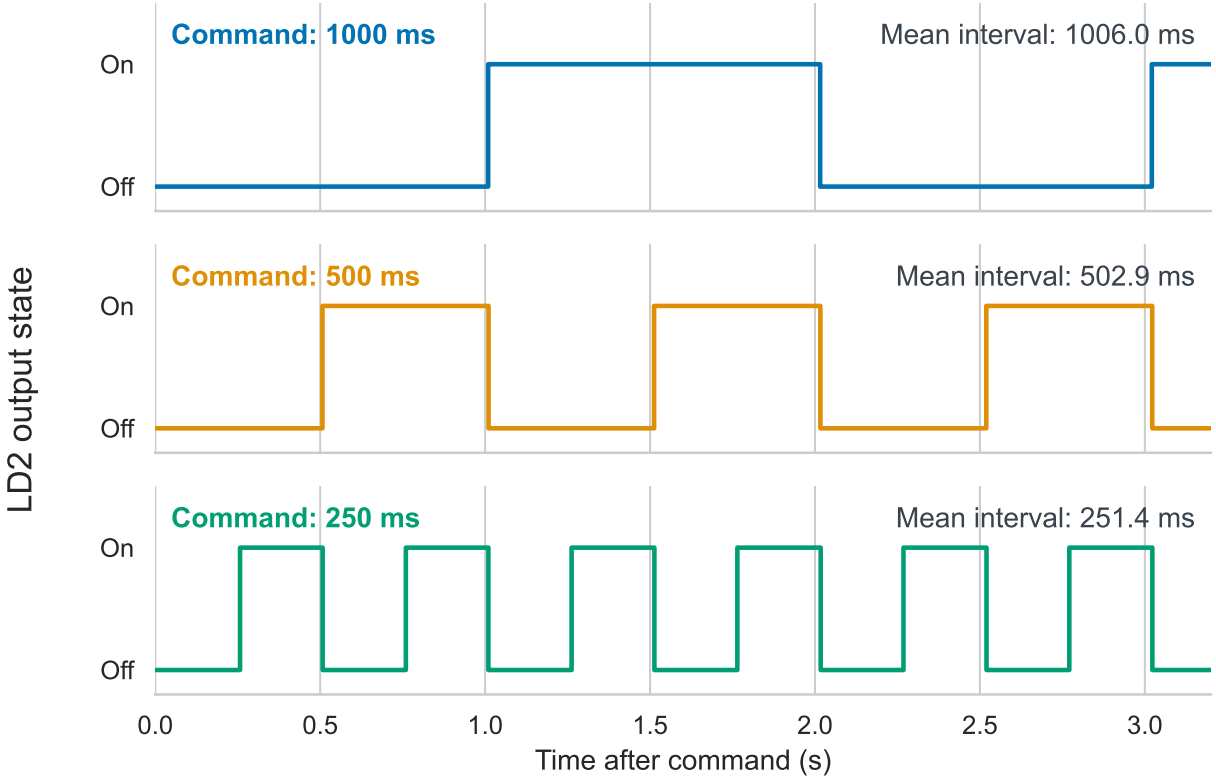


Figure 3: Recorded LD2 output states for three commanded toggle intervals.

Table 3: Measured inter-toggle intervals. SD denotes sample standard deviation.

Command (ms)	Mean (ms)	SD (ms)	Intervals
1000	1006.0	0.9	11
500	502.9	0.7	11
250	251.4	0.8	11

The mean interval scales with the requested delay and is approximately 0.6% above the nominal value in each case. A complete on/off cycle contains two toggles, so its period is $2T$. The counter is updated every ten toggles, corresponding to nominal update periods of 10, 5, and 2.5 s for the three commands.

4 Conclusion

The first task demonstrated priority-based response by applying the brake indicator before issuing its UART notification. The second task combined interrupt-driven reception with three queues to control LED timing and report counter and array results. Hardware output confirmed command parsing and queue communication, while measured intervals followed the requested 4:2:1 ratio. Allocating 1536 bytes to the LED task accommodated the 600-byte local array and its execution stack.